

NETWORK-BASED INTRUSION DETECTION SYSTEM

Deployment & Correlation Report

Suricata + Wazuh on Ubuntu

Prepared by	EMMANUEL ADEMOYEGA
Candidate ID	CA/DF1/233492
Lab Date	August 19, 2026

1. Objective

The objective of this lab was to deploy a network-based intrusion detection capability using Suricata and correlate the resulting network telemetry with Wazuh. The work focused on configuring detection rules, monitoring traffic, validating detections with controlled traffic from Kali Linux, and presenting the resulting events through the Wazuh dashboard.

- Set up a network-based intrusion detection system using Suricata.
- Configure rules and alerts to detect suspicious or malicious network activity.
- Monitor network traffic continuously for indicators of reconnaissance and protocol anomalies.
- Validate detection using controlled Nmap and ICMP traffic.
- Correlate Suricata JSON telemetry with Wazuh alerts and visualize the results.

Outcome The supplied evidence confirms that Suricata generated events in `/var/log/suricata/eve.json` and that Wazuh ingested, decoded and displayed those events under the `suricata` rule group.

2. Lab Architecture

The lab consisted of three logical components: a Wazuh Manager/Dashboard, an Ubuntu endpoint running both the Wazuh Agent and Suricata, and a Kali Linux system used to generate controlled test traffic. The addresses below are taken directly from the lab report and supplied screenshots.

Role	OS / Software	IP Address
Wazuh Manager + Dashboard	Wazuh v4.14.6 OVA	10.100.41.162
Monitored Endpoint	Ubuntu Desktop 24.04 + Wazuh Agent + Suricata	10.100.41.157
Attacker / Test Machine	Kali Linux	10.100.41.163

Traffic path: Kali Linux (10.100.41.163) → Ubuntu endpoint (10.100.41.157) → Suricata sensor → `/var/log/suricata/eve.json` → Wazuh Agent → Wazuh Manager/Dashboard (10.100.41.162).

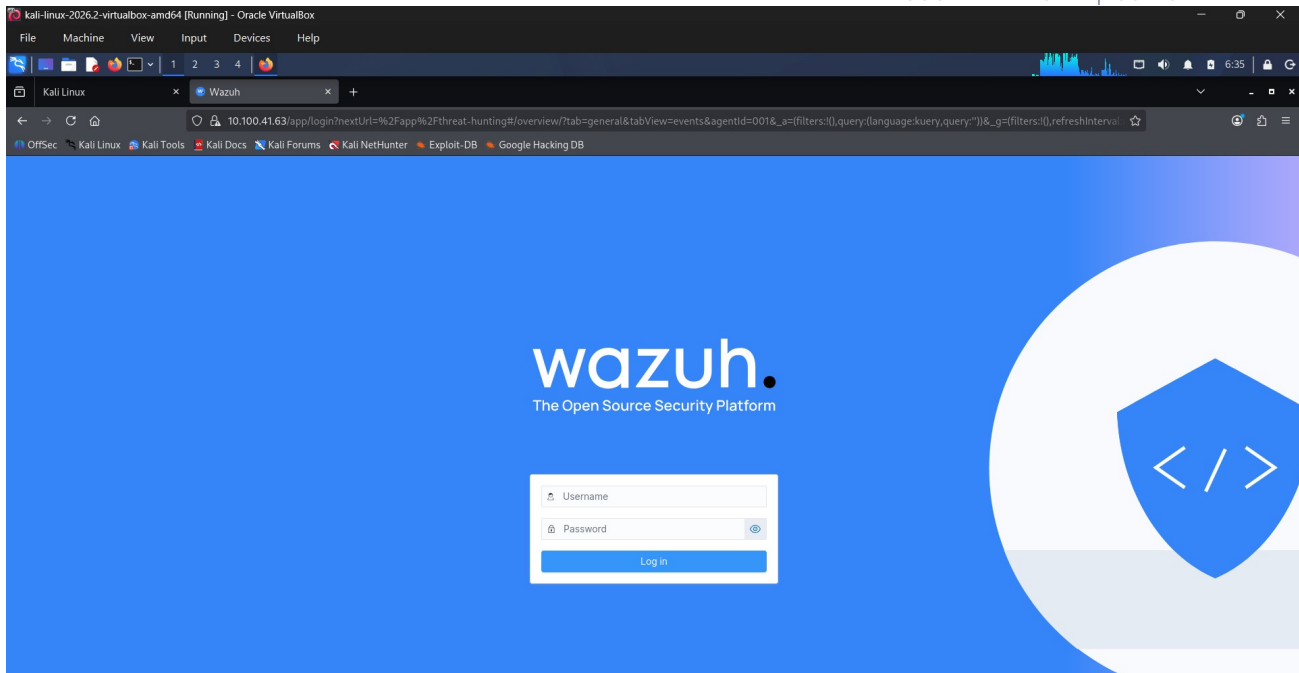
Note: The screenshots show the monitored interface as `enp0s3` and the Wazuh agent as Tony-ventures (Agent ID 001).

3. Installation & Configuration

3.1 Wazuh Server Installation

The original lab procedure used the Wazuh installation assistant to deploy the server and dashboard.

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh
sudo bash wazuh-install.sh -a
```



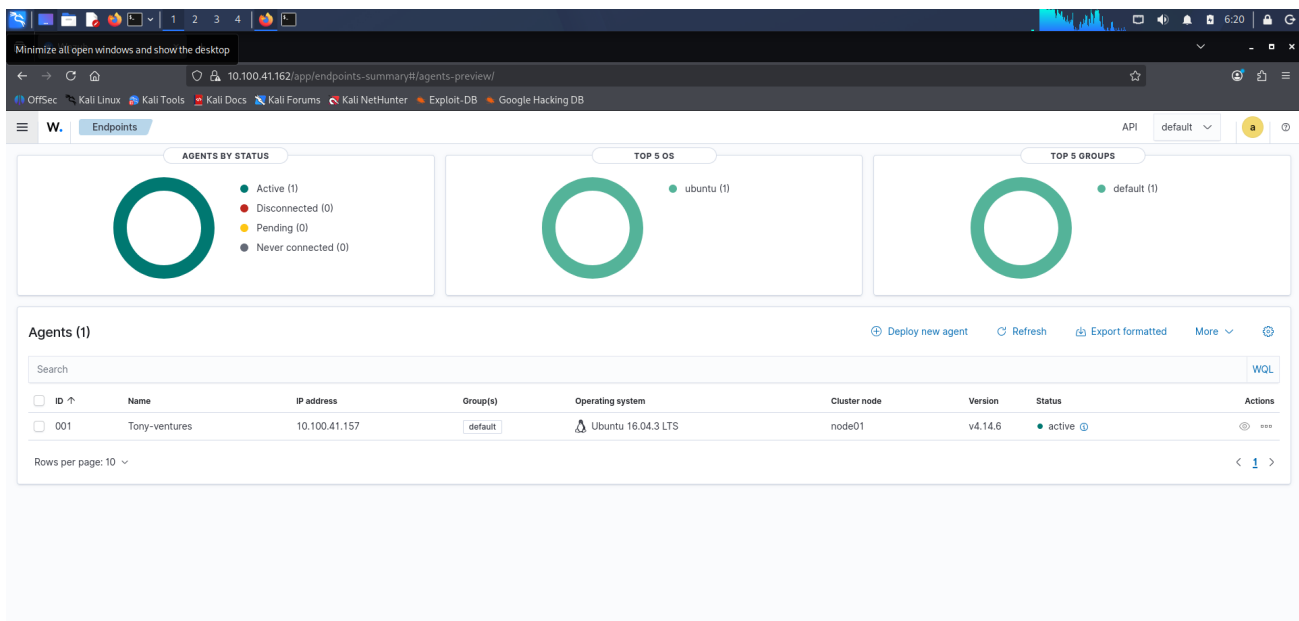
. Wazuh dashboard login page confirming successful dashboard deployment.

The dashboard screenshot provides visual confirmation that the Wazuh interface was reachable after installation.

3.2 Wazuh Agent Enrollment on Ubuntu Endpoint

The Ubuntu endpoint was enrolled as a Wazuh agent so that endpoint telemetry and Suricata JSON alerts could be forwarded to the manager.

```
curl -sO https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.9.0-1_amd64.deb
sudo WAZUH_MANAGER='<wazuh-server-ip>' dpkg -i ./wazuh-agent_4.9.0-1_amd64.deb
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```



Wazuh Agents view showing the enrolled Tony-ventures endpoint as active.

Version note The lab architecture identifies the manager/dashboard as Wazuh v4.14.6, while the documented agent command references the 4.9.0 package. This version difference is retained because it appears in the supplied report; production deployment should standardize compatible component versions.

3.3 Suricata Installation

Suricata was installed on the Ubuntu endpoint and configured to operate as the network IDS sensor.

```
sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt-get update
sudo apt-get install suricata -y
```

The Emerging Threats Open ruleset was then downloaded, extracted and placed in the Suricata rules directory.

```
cd /tmp/
curl -LO https://rules.emergingthreats.net/open/suricata-<version>/emerging.rules.tar.gz
sudo tar -xvzf emerging.rules.tar.gz
sudo mkdir -p /etc/suricata/rules
sudo mv rules/*.rules /etc/suricata/rules/
```

3.4 Suricata Configuration (Rules & Alerts)

The Suricata configuration was aligned with the monitored Ubuntu endpoint. The supplied report records the following key settings:

Setting	Value used	Purpose
HOME_NET	[10.100.41.157]	Defines the protected endpoint/network monitored by Suricata.
EXTERNAL_NET	any	Treats other traffic as external/untrusted.
default-rule-path	/etc/suricata/rules	Location of active detection rules.
rule-files	*.rules	Loads the downloaded rule files.
af-packet.interface	enp0s3	Network interface used by Suricata to inspect traffic.

```

GNU nano 2.5.3 File: /etc/suricata/suricata.yaml Modified
%YAML 1.1
---
# Suricata configuration file. In addition to the comments describing all
# options in this file, full documentation can be found at:
# https://suricata.readthedocs.io/en/latest/configuration/suricata-yaml.html
##
## Step 1: Inform Suricata about your network
##
vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[10.100.41.165]"
    #HOME_NET: "[192.168.0.0/16]"
    #HOME_NET: "[10.0.0.0/8]"
    #HOME_NET: "[172.16.0.0/12]"
    #HOME_NET: "any"

    EXTERNAL_NET: "!$HOME_NET"
    #EXTERNAL_NET: "any"

    HTTP_SERVERS: "$HOME_NET"
    SMTP_SERVERS: "$HOME_NET"
    SQL_SERVERS: "$HOME_NET"
    DNS_SERVERS: "$HOME_NET"
    TELNET_SERVERS: "$HOME_NET"
    AIM_SERVERS: "$EXTERNAL_NET"

```

Suricata configuration showing the protected network and interface settings.

```

sudo systemctl enable suricata
sudo systemctl start suricata
sudo systemctl status suricata

```

```

marlinspike@Lankey:~$ sudo systemctl status suricata
[sudo] password for marlinspike:
● suricata.service - LSB: Next Generation IDS/IPS
   Loaded: loaded (/etc/init.d/suricata; bad; vendor preset: enabled)
   Active: active (running) since Tue 2026-08-18 05:40:23 EDT; 2h 14min ago
     Docs: man:systemd-sysv-generator(8)
  Process: 1320 ExecStart=/etc/init.d/suricata start (code=exited, status=0/SUCCESS)
    CGroup: /system.slice/suricata.service
            └─1386 /usr/bin/suricata -c /etc/suricata/suricata.yaml --pidfile /var/run/suricata.pid --af-packet -D -vvv

Aug 18 05:40:22 Lankey systemd[1]: Starting LSB: Next Generation IDS/IPS...
Aug 18 05:40:23 Lankey suricata[1320]: Starting suricata in IDS (af-packet) mode... done.
Aug 18 05:40:23 Lankey systemd[1]: Started LSB: Next Generation IDS/IPS.
marlinspike@Lankey:~$

```

Suricata service status showing the IDS process running.

The supplied terminal evidence shows the Suricata service enabled and active, with the sensor operating against the configured network interface.

3.5 Forwarding Suricata Logs to Wazuh (Correlation Setup)

The Wazuh Agent was configured to monitor Suricata's eve.json output as JSON. This is the key correlation point between network-layer detections and Wazuh's SIEM view.

```

<ossec_config>
  <localfile>
    <log_format>json</log_format>
    <location>/var/log/suricata/eve.json</location>
  </localfile>
</ossec_config>

```

```
sudo systemctl restart wazuh-agent
```

In operation, Wazuh receives the Suricata JSON records, decodes fields such as source IP, destination IP, signature and severity, and exposes the resulting events in the Wazuh alert index. The supplied

alert details repeatedly show /var/log/suricata/eve.json as the input location and suricata as the rule group.

4. Testing Methodology

Detection was validated using controlled network activity from the Kali Linux test machine. The supplied report specifies Nmap scanning and ICMP traffic as the primary validation methods, and the Wazuh evidence confirms that the generated traffic produced Suricata-backed alerts.

Test	Observed method / traffic	Observed or expected detection
Nmap scan	nmap -A -p- -T4 against the Ubuntu endpoint	ET SCAN Nmap Scripting Engine User-Agent Detected; Potential SSH Scan; application-layer protocol alerts; web-server 400/501 responses.
ICMP test	Ping traffic from Kali to the monitored endpoint	GPL ICMP_INFO PING *NIX and related ICMP/Suricata telemetry.

```

(Toyin@Lankey) ~
$ ping -c 4 10.100.41.157
PING 10.100.41.157 (10.100.41.157) 56(84) bytes of data.
64 bytes from 10.100.41.157: icmp_seq=1 ttl=64 time=10.2 ms
64 bytes from 10.100.41.157: icmp_seq=2 ttl=64 time=2.95 ms
64 bytes from 10.100.41.157: icmp_seq=3 ttl=64 time=2.78 ms
64 bytes from 10.100.41.157: icmp_seq=4 ttl=64 time=2.35 ms

--- 10.100.41.157 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 2.354/6.072/16.204/5.853 ms

(Toyin@Lankey) ~
$ nmap -A -p- -T4 10.100.41.157
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 08:40 -0400
Nmap scan report for 10.100.41.157
Host is up (0.010s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
|_ ssh-hostkey:
|_ 2048 d6:01:90:39:2d:8f:46:fb:03:86:73:b3:3c:54:7e:54 (RSA)
|_ 256 f1:f3:c8:dd:ba:a4:b5:f7:13:9a:da:3a:bb:4d:93:04 (ECDSA)
|_ 256 12:42:98:d2:a3:e7:36:4f:be:0b:ce:36:0b:7e:0d:9e (ED25519)
|_ 80/tcp open  http  Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.18 (Ubuntu)
MAC Address: 08:00:27:41:CE:84 (Oracle VirtualBox virtual NIC)
Device type: general purpose router
Running: Linux 4.X|5.X, Mikrotik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), Mikrotik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT ADDRESS
1 10.26 ms 10.100.41.157
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.

```

Kali Linux terminal evidence showing the network reconnaissance test used in the lab.

The Nmap evidence is especially useful because the Wazuh alert details expose the originating address, destination address, destination port, HTTP method and Nmap scripting-engine user agent, allowing the alert to be validated against the test source.

5. Results Detection & Correlation

The supplied Wazuh evidence demonstrates successful end-to-end detection and correlation. With the dashboard filtered to rule.groups:suricata, Wazuh displayed 45 hits for the Aug. 14–21, 2026 window. The activity was concentrated around Aug. 18, when the Nmap test generated multiple alerts in rapid succession.

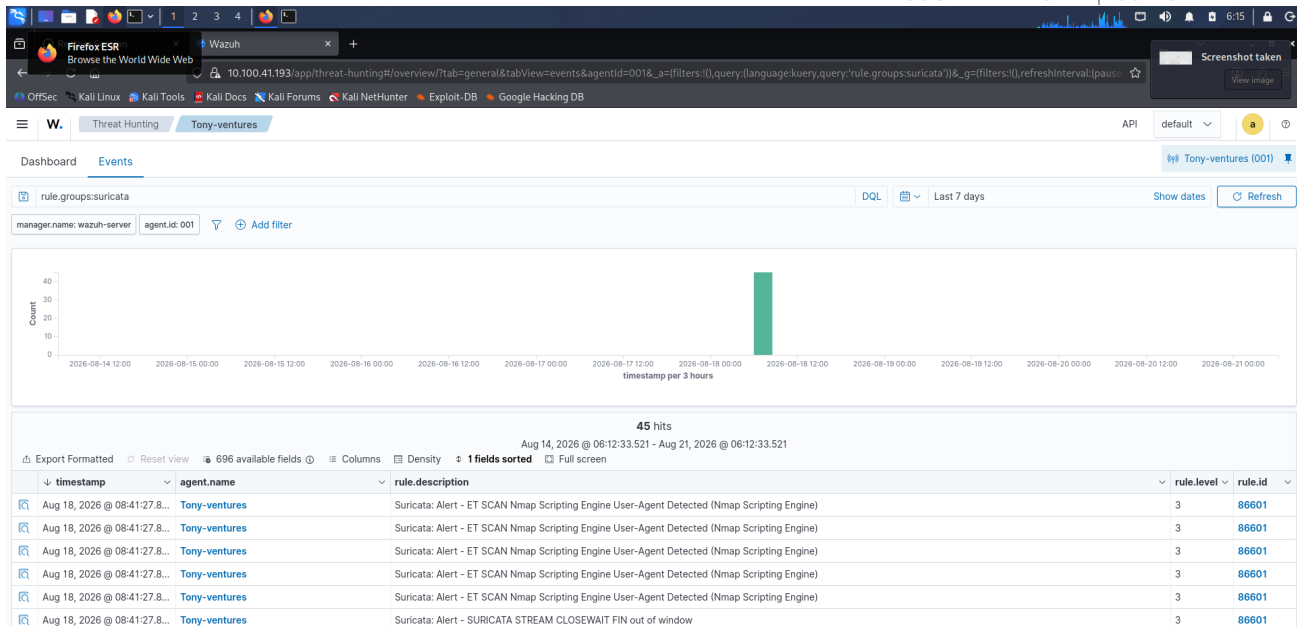
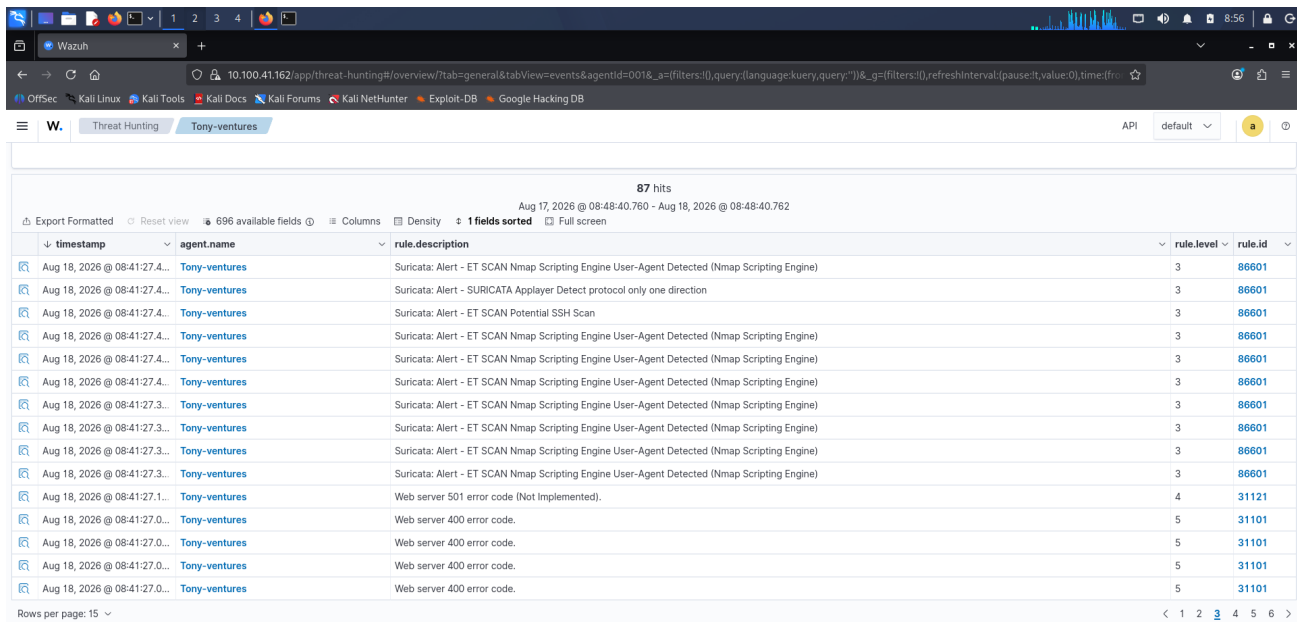


Figure 6. Wazuh Threat Hunting view filtered to rule.groups:suricata, showing 45 Suricata-related hits.

The alert list contains repeated ET SCAN Nmap Scripting Engine User-Agent Detected events (rule 86601, level 3), alongside SURICATA STREAM CLOSEWAIT FIN out of window. The separate Nmap-focused view records 87 hits between Aug. 17 and Aug. 18 and includes Potential SSH Scan, application-layer protocol detection and web-server error responses.



Nmap-focused Wazuh event view showing correlated Suricata and web-server alerts.

An expanded event confirms the correlation chain: the source was 10.100.41.163, the destination was 10.100.41.157 on TCP/80, the HTTP method was OPTIONS, and the user agent identified Nmap Scripting Engine. The event originated from /var/log/suricata/eve.json and was mapped to rule 86601 at level 3.

The screenshot shows the Wazuh Threat Hunting interface. The main table displays a list of events with columns for timestamp, agent name, and rule description. The details panel on the right shows the following fields:

Field	Value
data.app_proto	http
data.dest_ip	10.100.41.157
data.dest_port	80
data.event_type	alert
data.flow.bytes_toclient	387
data.flow.bytes_to_server	486
data.flow.packets_toclient	3
data.flow.packets_to_server	4
data.flow.start	2026-08-18T08:41:26.787905-0400
data.flow_id	511751040140737.000000
data.http.hostname	10.100.41.157
data.http.http_content_type	text/html
data.http.http_method	OPTIONS
data.http.http_user_agent	Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)
data.http.length	0
data.http.protocol	HTTP/1.1
data.http.status	200
data.http.url	/
data.in_iface	enp0s3
data.proto	TCP
data.src_ip	10.100.41.163

Expanded Suricata event fields for the Nmap HTTP request.

The screenshot shows the Wazuh Threat Hunting interface. The main table displays a list of events with columns for timestamp, agent name, and rule description. The details panel on the right shows the following fields:

Field	Value
data.http.status	200
data.http.url	/
data.in_iface	enp0s3
data.proto	TCP
data.src_ip	10.100.41.163
data.src.port	60192
data.timestamp	Aug 18, 2026 @ 08:41:26.799
data.tx_id	0
decoder.name	json
id	1787056887.99574
input.type	log
location	/var/log/suricata/eve.json
manager.name	wazuh-server
rule.description	Suricata: Alert - ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)
rule.firedtimes	40
rule.groups	ids, suricata
rule.id	86601
rule.level	3
rule.mail	false
timestamp	Aug 18, 2026 @ 08:41:27.839

Wazuh event metadata showing the Suricata signature, source IP, rule ID and fired count.

The ICMP test was also visible in Wazuh. The supplied document detail shows protocol ICMP, source 10.100.41.163, interface enp0s3 and the signature GPL ICMP_INFO PING *NIX, demonstrating that the sensor was able to observe and forward ICMP activity as well.

The screenshot displays the Wazuh Threat Hunting interface. On the left, a table lists alerts with columns for timestamp, agent name, and rule description. The right pane shows the 'Document Details' for a selected event, including fields like data.flow_id, data.icmp_code, data.icmp_type, data.in_iface, data.proto, data.src_ip, data.src_port, data.timestamp, decoder.name, id, input.type, location, manager.name, rule.description, rule.firedtimes, rule.groups, rule.id, rule.level, rule.mail, and timestamp.

. Expanded Wazuh event for the ICMP test, sourced from Suricata eve.json.

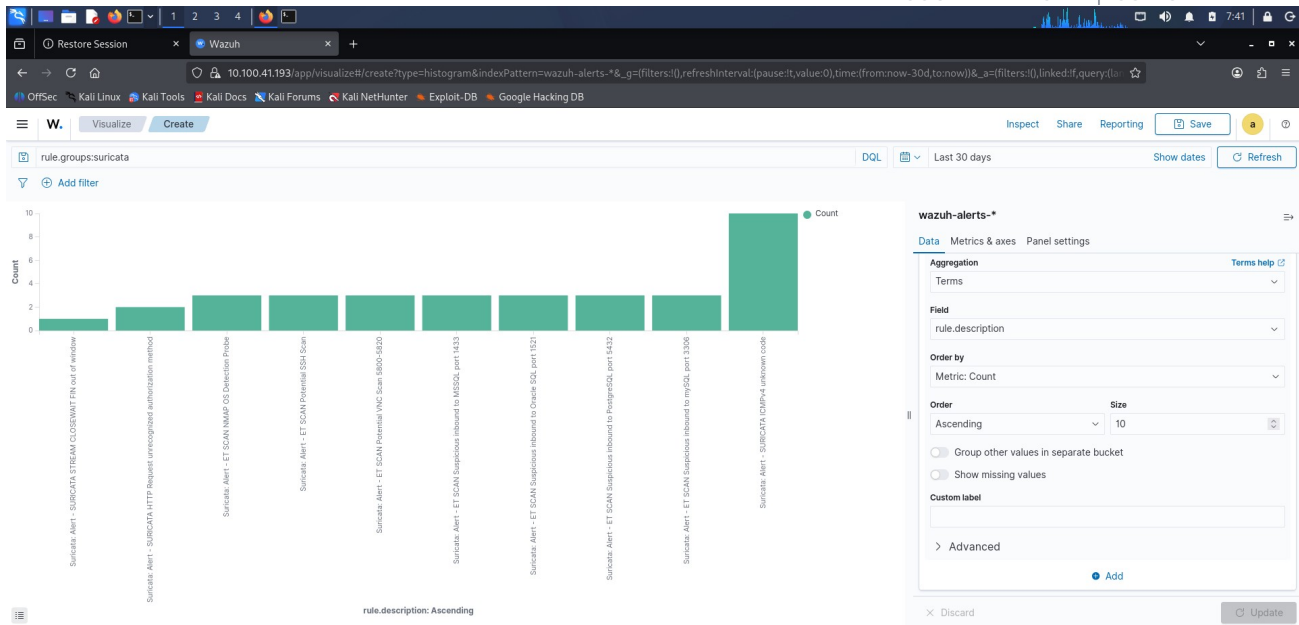
5.1 Alert Summary

Time / Window	Signature / Rule	Level	Source → Destination	Disposition
18 Aug 2026 ~08:41:27	ET SCAN Nmap Scripting Engine User-Agent Detected (86601)	3	10.100.41.163 → 10.100.41.157:80	Logged / correlated
18 Aug 2026 ~08:41:27	ET SCAN Potential SSH Scan (86601)	3	10.100.41.163 → monitored endpoint	Logged / correlated
18 Aug 2026 ~08:41:27	SURICATA Applayer Detect protocol only one direction (86601)	3	10.100.41.163 → monitored endpoint	Logged / correlated
18 Aug 2026 ~08:41:27	SURICATA STREAM CLOSEWAIT FIN out of window (86601)	3	10.100.41.163 → 10.100.41.157	Logged / correlated
18 Aug 2026 ~08:39:49	GPL ICMP_INFO PING *NIX (86601)	3	10.100.41.163 → monitored endpoint	Logged / correlated
18 Aug 2026 ~08:41:27	Web server 400 error code (31101)	5	Nmap test traffic → web service	Logged
18 Aug 2026 ~08:41:27	Web server 501 error code (31121)	4	Nmap test traffic → web service	Logged

Detection assessment The evidence supports successful detection and SIEM correlation. It does not by itself prove that the traffic represented a successful compromise; the observed Nmap and ICMP activity is best treated as reconnaissance/test traffic within the controlled lab.

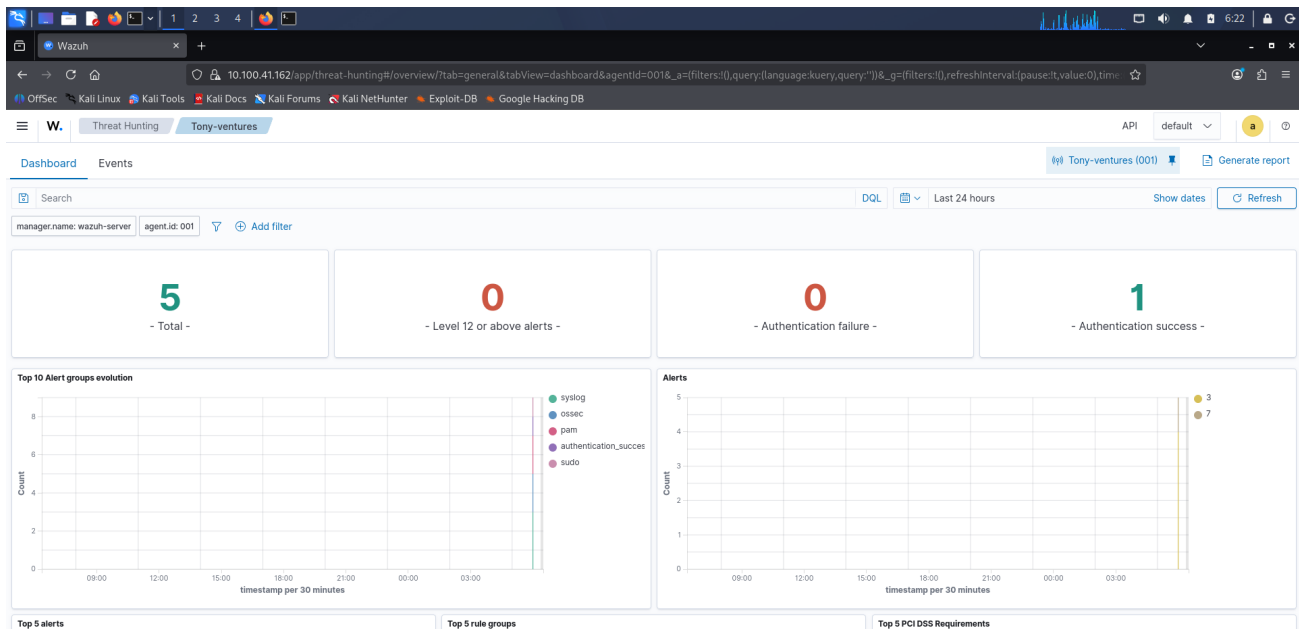
6. Visualization

A custom Wazuh visualization was created using the wazuh-alerts-* index pattern, filtered to rule.groups:suricata. The visualization uses a Terms aggregation on rule.description and Count as the metric, allowing the most frequent Suricata signatures to be compared directly.



Custom Wazuh visualization of Suricata alert signatures using rule.description and event count.

The visualization shows the relative frequency of the observed signatures over the selected 30-day window. The most prominent category is the SURICATA ICMPv4 unknown code signature, while several scan and protocol-related signatures appear at lower but repeated counts. This view is useful for identifying recurring alert types and prioritizing rule tuning.



Wazuh dashboard overview for the last 24 hours showing five total alerts, no level-12-or-above alerts, no authentication failures and one authentication success.

The dashboard overview provides useful operational context beyond the Suricata-only view. In the selected 24-hour window, Wazuh displayed five total alerts, zero level-12-or-above alerts, zero authentication failures and one authentication success. Because this dashboard uses a different time window and includes broader alert groups, these values should not be directly compared with the 45 Suricata hits from the seven-day Suricata-filtered view.

7. Response Mechanisms

The supplied report includes a Wazuh active-response example using firewall-drop for high-severity Suricata alerts. This report treats the configuration below as the recommended response mechanism rather than executed evidence.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_group>suricata</rules_group>
  <level>10</level>
</active-response>
```

For this lab, the defensible response workflow is therefore: detect → validate the source and signature → assess severity and context → block or contain when justified → preserve the alert evidence → document the action. In a production SOC, the firewall-drop action should be tested carefully against false-positive scenarios before being enabled for broad Suricata rule groups.

8. Challenges & Resolutions

The supplied material contains several documentation and operational points that should be made explicit for a professional handover.

Challenge / observation	Resolution / handling	Evidence
Component version difference: Wazuh manager/dashboard is recorded as v4.14.6 while the agent installation command references 4.9.0.	Retain the documented procedure for traceability, but standardize compatible Wazuh component versions before production deployment.	Lab report + installation section.
Correct packet-capture interface must be used by Suricata.	The configuration/evidence identifies enp0s3 as the active interface, and Suricata telemetry records enp0s3.	Suricata configuration and expanded event screenshots.
Large numbers of repeated scan/protocol alerts can make triage noisy.	Use the Wazuh visualization and rule/signature counts to identify recurring signatures and tune rules or thresholds where appropriate.	45-hit Suricata view + custom visualization.
Automated response is not evidenced in the supplied screenshots.	Document the active-response block as a recommended control and keep detection/correlation as the confirmed outcome.	Response section and supplied dashboard evidence.

A key strength of the lab is that the evidence does not stop at a single alert. The screenshots connect the traffic source, Suricata JSON log, Wazuh decoder, rule group, rule ID and dashboard visualization, making the detection path auditable.

9. Conclusion

This lab successfully demonstrates the core workflow of a network-based intrusion detection and SIEM correlation pipeline. Suricata monitored the Ubuntu endpoint, generated JSON events in eve.json, and the Wazuh Agent forwarded those events to the Wazuh Manager for decoding, rule matching and visualization. Controlled Nmap and ICMP traffic from Kali Linux produced observable

alerts, including Nmap scripting-engine detection, potential SSH scanning, application-layer protocol anomalies, web-server error responses and ICMP telemetry.

The Wazuh evidence also demonstrates the value of correlation and visualization: analysts can move from a high-level alert count to an individual document and inspect source/destination IPs, ports, protocols, HTTP attributes, Suricata signatures and Wazuh rule metadata. This provides a practical foundation for SOC triage and incident investigation.

The main limitations are the controlled lab scale, the possibility of rule noise, and the absence of supplied evidence for an executed automated containment action. Future improvements should include standardized component versions, additional log sources, tuned detection rules, controlled active-response testing, and integration with a ticketing or notification workflow.

References

- **Wazuh Documentation — Network IDS Integration (Suricata)** — <https://documentation.wazuh.com/>
- **Suricata Documentation** — <https://suricata.io/>
- **Emerging Threats Open Ruleset** — <https://rules.emergingthreats.net/>